Méthodes d'agrégation avec Python

Octobre 2026

Cédric Dangeard
cedric.dangeard@orange.com



Cours 3 : Au programme

- Compléments sur les arbres de décision
- Finir TP 2: Arbres de décision (sklearn)
- Les méthodes d'agrégation
  - Bagging (Random Forests)
  - Boosting (AdaBoost, Gradient Boosting, XGBoost)
- Sur-apprentissage et validation croisée (rappels)
- Métriques de performance
- TP : Bagging et Boosting (sklearn)

Arbres de décisions : Rappels

- Partition récursive de l'espace des features
- Critère de séparation : Gini, Entropie, SSR (somme des carrés des résidus intra-nœud)
- Arbres de classification et de régression
- Arbres peu profonds : Biais élevé, Variance faible
- Arbres profonds : Biais faible, Variance élevée

On dispose de données (x_i, y_i), avec $x_i \in \mathbb{R}^p$.
Un nœud t contient un ensemble de données S_t, d'effectif n_t.
On suppose ici que toutes les observations ont le même poids.

Une coupure choisit une variable j et un seuil s :

$$S_L(j, s) = \{i \in S_t : x_{ij} \leq s\}, \qquad S_R(j, s) = \{i \in S_t : x_{ij} > s\}$$

- Pour une variable numérique, on teste les milieux entre valeurs distinctes consécutives triées dans le nœud.
- On ne garde que les coupures respectant les contraintes, notamment `min_samples_leaf`.

Pour une impureté I, avec $n_L + n_R = n_t$:

$$\Delta I(t, j, s) = I(t) - \frac{n_L}{n_t} I(L) - \frac{n_R}{n_t} I(R)$$

$$(j^*, s^*) \in \underset{(j,s)\ \text{admissibles}}{\arg\max} \Delta I(t, j, s)$$

- Les poids sont essentiels : une grande feuille compte davantage qu'une petite.
- On répète ce choix dans chaque enfant, jusqu'aux critères d'arrêt.
- C'est une optimisation **gloutonne** : la meilleure coupure locale ne garantit pas le meilleur arbre global.

Pour K classes, la proportion de la classe k dans le nœud t est :

$$p_{tk} = \frac{1}{n_t} \sum_{i \in S_t} \mathbf{1}_{\{y_i = k\}}$$

$$I_G(t) = 1 - \sum_{k=1}^{K} p_{tk}^2, \qquad H(t) = -\sum_{k=1}^{K} p_{tk} \log_2(p_{tk})$$

- Convention : $0 \log_2(0) = 0$; un nœud pur a une impureté nulle.
- Deux classes à parts égales : $I_G = 0{,}5$ et $H = 1$ bit.
- Les deux critères favorisent des enfants homogènes, mais peuvent choisir des coupures différentes.

Pour une nouvelle observation x, on suit les règles jusqu'à sa feuille $t(x)$:

$$\widehat{P}(Y = k \mid X = x) = p_{t(x),k}, \qquad \hat{y}(x) \in \arg\max_k p_{t(x),k}$$

- `predict_proba` renvoie les proportions de classes dans cette feuille.
- `predict` renvoie la classe majoritaire (égalité départagée par l'ordre des classes dans scikit-learn).
- Exemple : une feuille contenant 3 A et 1 B prédit A, avec des probabilités estimées $(0{,}75;\ 0{,}25)$.
- Ces fréquences ne garantissent pas des probabilités bien calibrées, surtout dans les petites feuilles.

Avec une perte quadratique, la meilleure constante dans un nœud est sa moyenne :

$$\arg\min_{c \in \mathbb{R}} \sum_{i \in S_t} (y_i - c)^2 = \bar{y}_t = \frac{1}{n_t} \sum_{i \in S_t} y_i$$

La dérivée par rapport à c vaut $2n_t(c - \bar{y}_t)$ et s'annule en $\bar{y}_t$.

$$\mathrm{SSR}(t) = \sum_{i \in S_t} (y_i - \bar{y}_t)^2, \qquad I(t) = \frac{\mathrm{SSR}(t)}{n_t}$$

Avec `criterion="squared_error"`, maximiser le gain revient à minimiser $\mathrm{SSR}(L) + \mathrm{SSR}(R)$. Chaque feuille prédit sa moyenne.

Pour un arbre T, avec $\mathcal{L}(T)$ l'ensemble de ses feuilles, appris sur N observations :

$$R(T) = \sum_{\ell \in \mathcal{L}(T)} \frac{n_\ell}{N} I(\ell), \qquad R_\alpha(T) = R(T) + \alpha \, |\mathcal{L}(T)|$$

- $R(T)$ mesure l'impureté pondérée des feuilles sur l'entraînement.
- $|\mathcal{L}(T)|$ est le nombre de feuilles ; $\alpha \geq 0$ pénalise la complexité.
- Le post-élagage cherche un sous-arbre minimisant R_α : augmenter `ccp_alpha` favorise un arbre plus petit.
- On choisit α par **validation croisée sur l'entraînement**, sans utiliser le jeu de test.

Arbres de décisions : Limites

- Variance élevé & surapprentissage pour les arbres trop profonds
- Peu performant en général

Solutions :

- Comment améliorer les performances des arbres ?

Bootstrap Aggregating

Soit X_i un ensemble de variables indépendantes de même loi dont la variance est $\sigma^2 < \infty$. La variance de la moyenne est :

$$Var(\frac{\sum_{i=1}^{n} X_i}{n}) = \frac{1}{n^2} \sum_{i=1}^{n} Var(X_i) = \frac{1}{n^2} n\sigma^2 = \frac{\sigma^2}{n}$$

Pour des variables de même variance σ^2, avec une corrélation ρ entre chaque paire :

$$Var(\frac{\sum_{i=1}^{n} X_i}{n}) = \frac{1-\rho}{n}\sigma^2 + \rho\sigma^2$$

Bootstrap Sampling

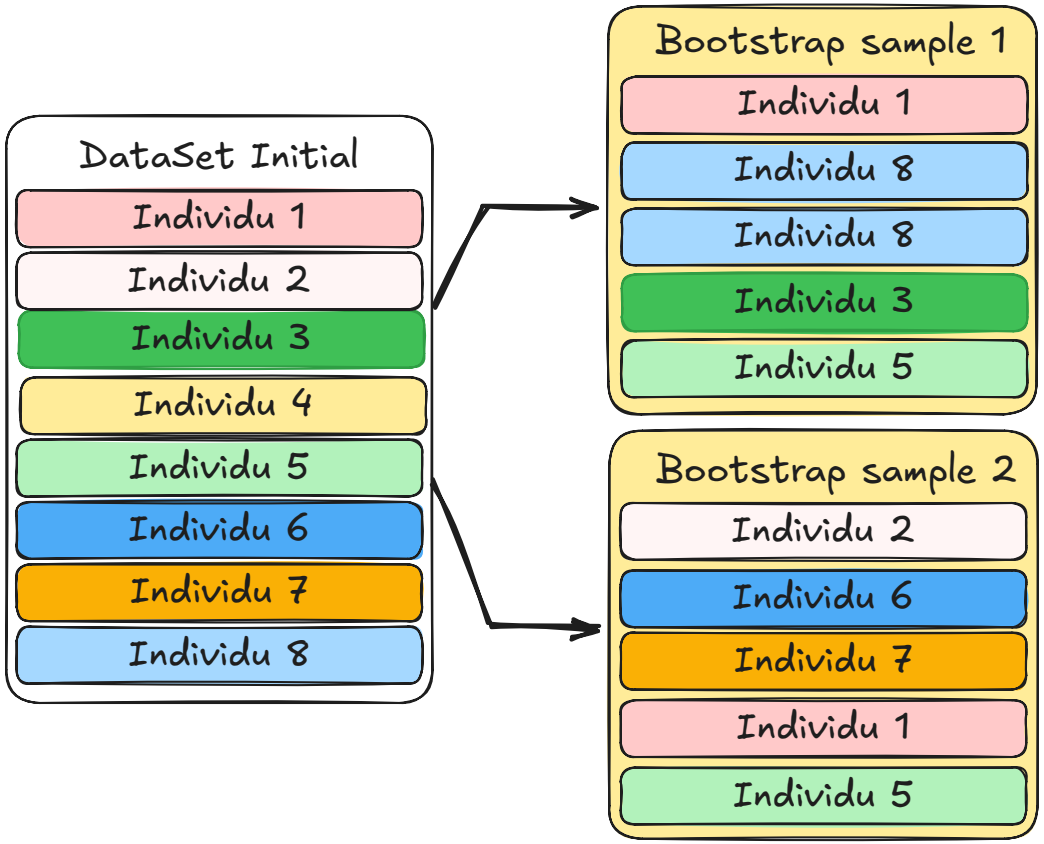
Objectif du bagging : réduire la variance d'un estimateur.

- Entraîner plusieurs modèles sur des rééchantillonnages du jeu d'entraînement, puis agréger leurs prédictions.
- Le bootstrap produit ces échantillons par **tirage avec remise** ; il ne crée pas de nouvelles observations.
- Les modèles restent corrélés : plus leurs erreurs sont similaires, moins l'agrégation réduit la variance.

Pour chaque modèle :

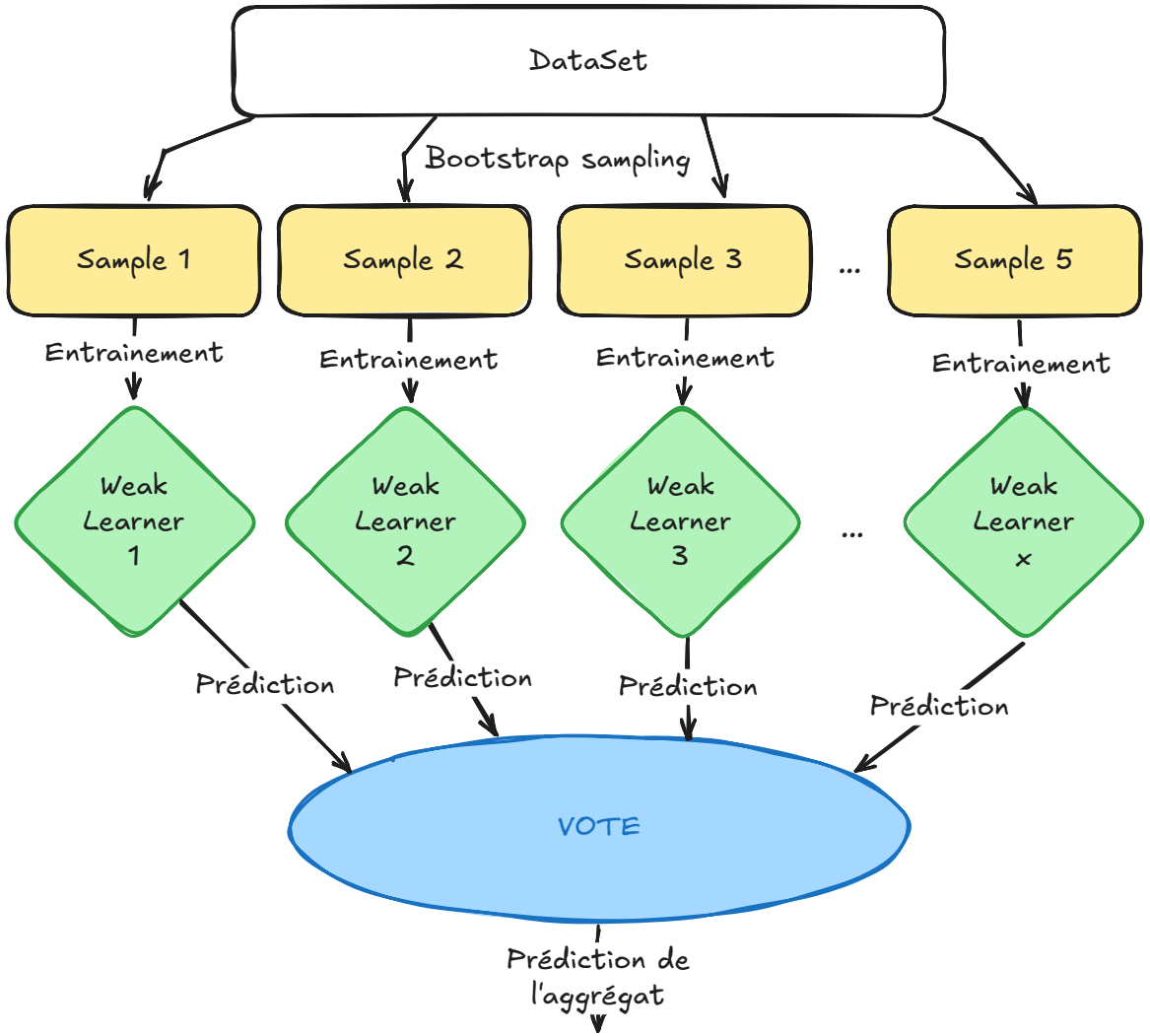
- Tirer N observations **avec remise** parmi les N observations d'entraînement.
- Certaines sont répétées, d'autres absentes de ce tirage.
- Entraîner un modèle sur cet échantillon bootstrap.

Répéter avec un nouveau tirage pour chaque modèle.



Aggregating

- Le bagging consiste donc à prendre la moyenne d'un ensemble de modèles
- On appelle en général ces modèles *Weak Learner*
- On peut utiliser tout types de modèles, idéalement des modèles faible biais /forte variance.



Random Forests

- Le Bagging appliqué aux arbres de décisions
- Chaque arbre est construit sur un échantillon bootstrapé des données
- A chaque nœud, on ne considère qu'un sous-ensemble aléatoire des variables
- Très robuste au sur-apprentissage
- Très performant en général
- Très facile à paralléliser
- Permet d'estimer l'importance des variables (cf cours 4)

Random Forest

- **n_estimators** : *(int)* Nombre d'arbres dans la forêt
- **max_features** : *(int/float/string)* Nombre de variables à considérer à chaque nœud
- **bootstrap** : *(bool)* Tirage avec remise des données

```python
from sklearn.ensemble import RandomForestClassifier

# Définition du modèle
rf = RandomForestClassifier(
    n_estimators=10,
    bootstrap=True,
    max_features='sqrt',
  n_jobs=-1
)

# Entrainement du modèle
rf.fit(X, y)
```

Bagging sur Sklearn : BaggingClassifier

- **max_features** / **max_samples** : *(float/int)*
  Nombre d'individus et de variables tirés avec remise
- **bootstrap_features** / **bootstrap** :*(bool)*:
  Tirage avec remise ou non des variables/données.
- **oob_score** *(bool)* : Si **bootstrap** erreur out of bag ou

```python
from sklearn.ensemble import BaggingClassifier
from sklearn.neighbors import KNeighborsClassifier

#Definition du modèle
bagging = BaggingClassifier(
  estimator=KNeighborsClassifier(),
    n_estimators=10,
    max_samples=0.5,
    max_features=0.5,
    bootstrap=True,
    bootstrap_features=False,
    oob_score=False,
    n_jobs=-1,
)

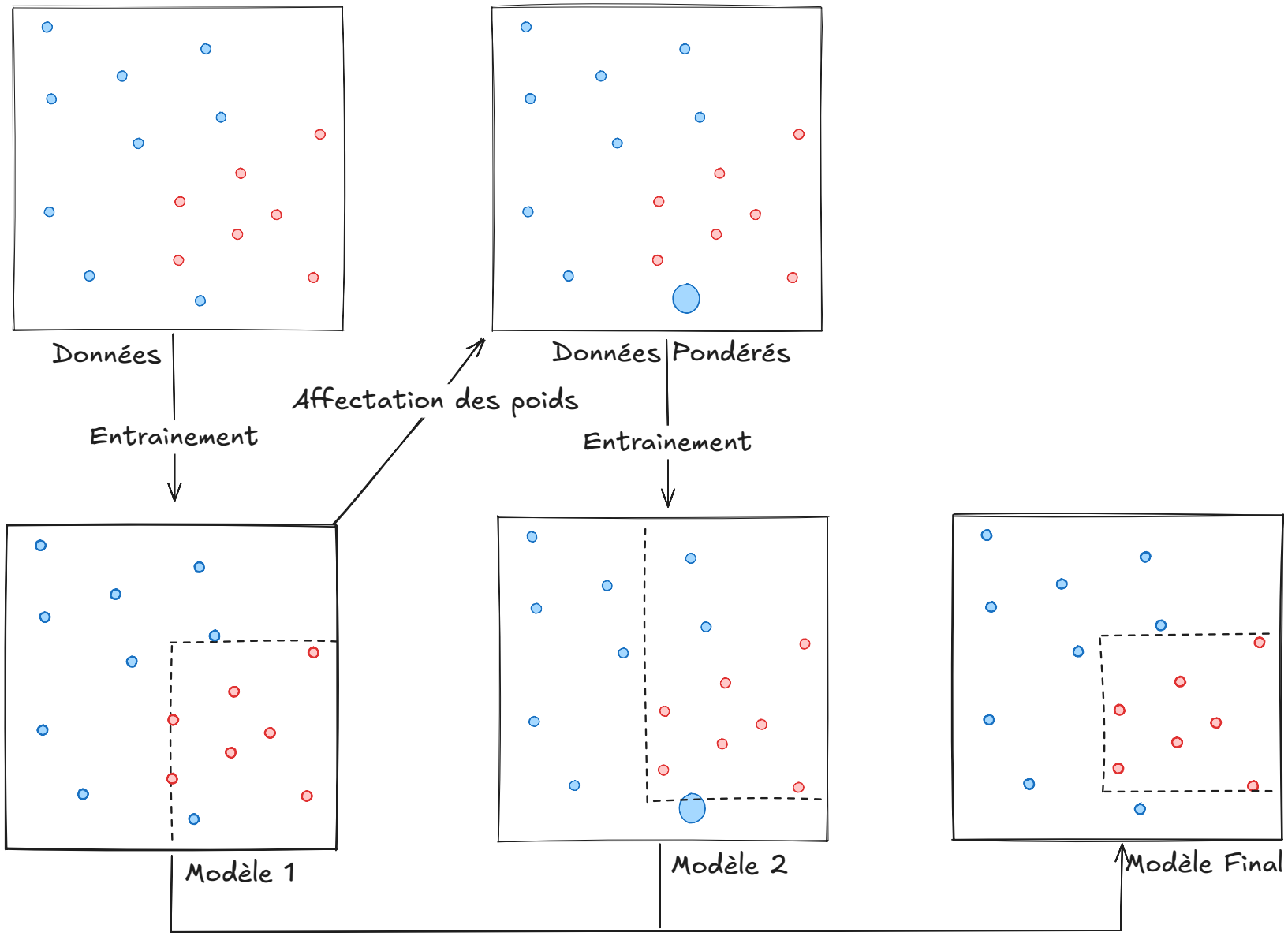
# Entrainement du modèle
bagging.fit(X, y)
```

Boosting

A l'instar du Bagging, concept du Boosting est d'améliorer la performance de "weak Learners".

Cependant, cette fois on procède de façon itérative, chaque modèle viendra renforcer les faiblesses du modèles précédent.

Il existe plusieurs algorithmes pour les problèmes de régression et de classification. Les plus utilisés sur python sont : adaboost, gradient boosting, XGBoost


Données
Entrainement
Modèle 1
Affectation des poids
Données Pondérés
Entrainement
Modèle 2
Modèle Final

L'algorithme d'Adaptive Boosting

- On procède par itérations successives
- A chaque itération on va chercher à corriger les erreurs du modèle précédent
- On va pour cela pondérer les individus mal classés
- On va aussi pondérer les modèles en fonction de leur performance
- Le modèle final est une combinaison linéaire des modèles intermédiaires

Algorithme d'Adaboost :

h^s : modèle à l'itération s
w_i^s : poids de l'individu i à l'itération s
y_i : label de l'individu i (+1 ou -1)
x_i : features de l'individu i

- On initialise de façon **uniforme** les poids de chaque individu i : w_i^0
- Pour chaque itération s :
  - On choisis le modèle qui minimise l'erreur en fonction de w_i^s.
  - On utilise le modèle h^s pour calculer le taux d'erreur E^s

- On calcule : $\alpha^s = \frac{1}{2}\ln\left(\frac{1-E^s}{E^s}\right)$
  - α^s est une mesure de la performance du modèle h^s
  - Plus E^s est petit, plus α^s est grand
- On met à jour les poids: $w_i^{s+1} = w_i^s . \exp^{-\alpha^s y_i h^s(x_i)}$
  - Si h^s classe correctement x_i, w_i^{s+1} diminue (car $y_i h^s(x_i) > 0$)
  - Sinon w_i^{s+1} augmente
  - On normalise les poids w_i^{s+1}
- Condition d'arret : $s >$ max_iter (dans sklearn : `n_estimators`)
- Le modèle final $H(x) = signe(\sum_{i=1}^{n} \alpha^i h^i(x))$

Adaboost sur Sklearn

estimator : *(Estimator)*:
défaut : **DecisionTreeClassifier**
n_estimators : *(int)*:
Nombre d'itérations maximale
learning_rate : *(float)*:
coefficient devant α^s lors de la mise à jour des poids
généralement entre 0.01 et 1.0, plus petit = plus d'itérations

```python
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
#Definition du modèle
ada = AdaBoostClassifier(
  estimator=DecisionTreeClassifier(max_depth=1),
    learning_rate=1.0,
    n_estimators=50
)

# Entrainement du modèle
ada.fit(X, y)
```

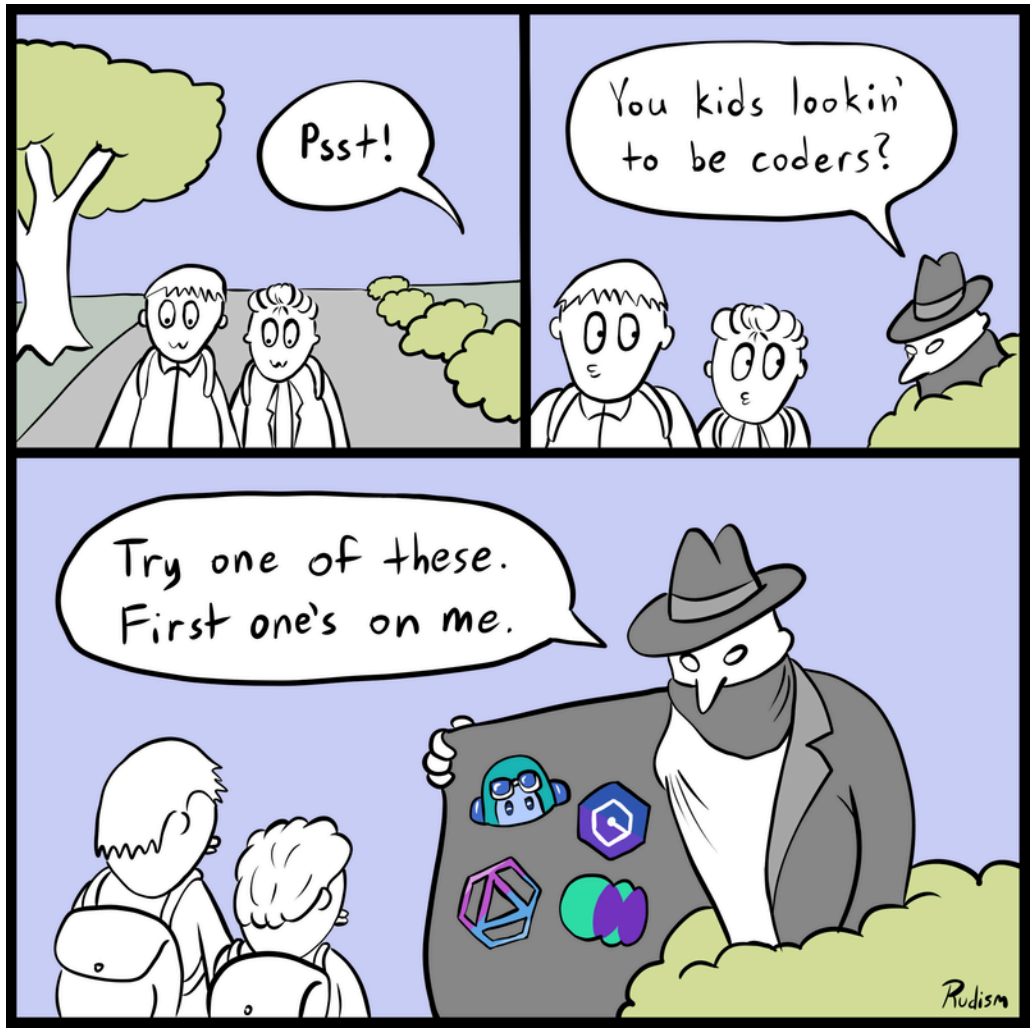
Gradient Boosting

L'idée du boosting est de prédire itérativement les résidus du modèle précedent.

On va utiliser une fonction de perte type (expodentielle, log-loss).

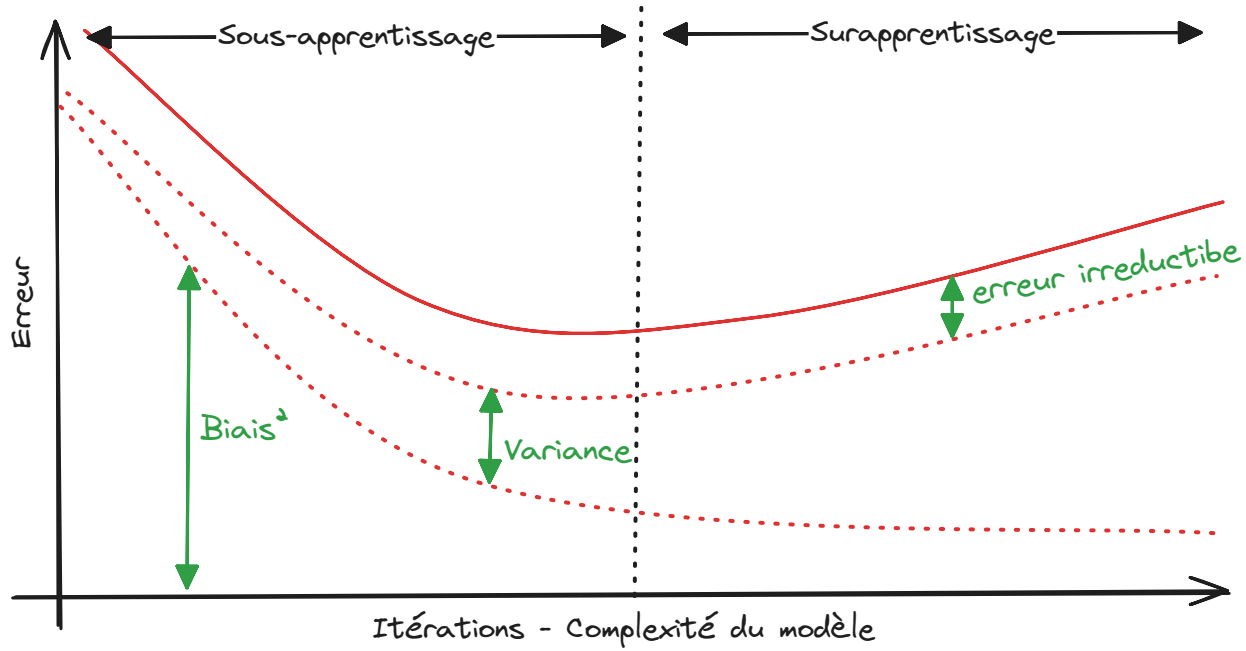
Et chercher à prédire de façon additive les résidus laissés par les modèles précédents.

TP : Aggrégations



Surapprentissage (rappels)

$$\vec{E}^2_{new} = \underbrace{\mathbb{E}[\hat{f}(x_*) - f(x_*)]^2}_{Biais^2} + \underbrace{\mathbb{E}[(\hat{f}(x_*) - \mathbb{E}[\hat{f}(x_*)])^2]}_{Variance} + \underbrace{\sigma^2}_{\substack{erreur \\ irreductible}}$$



Indicateurs de performance

- Accuracy (Exactitude)

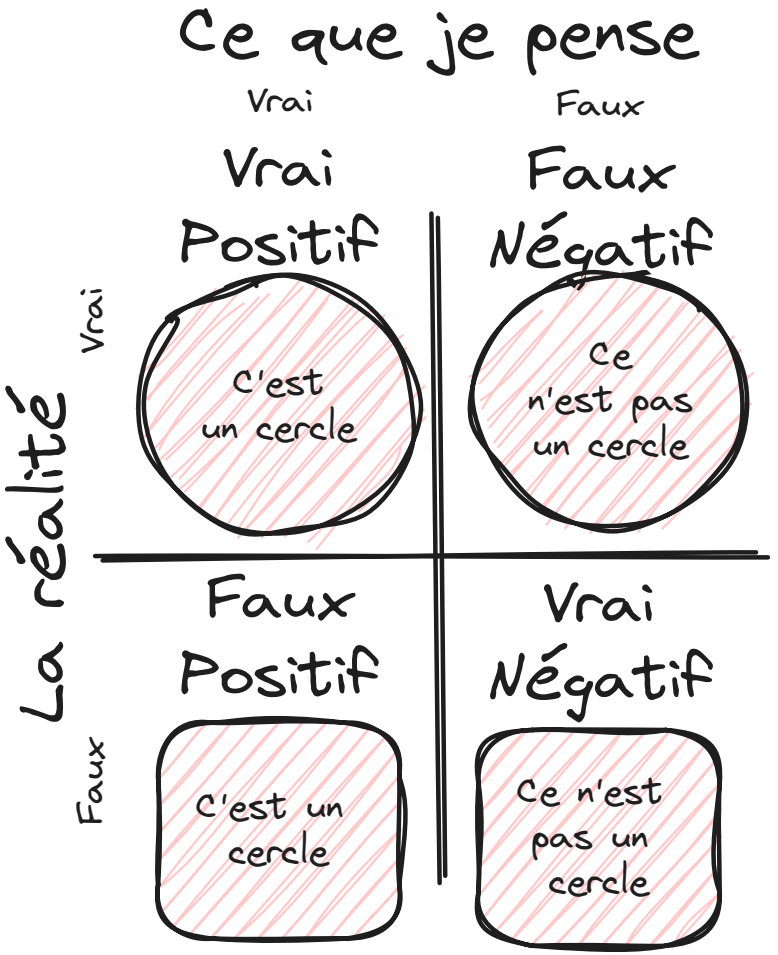
$$Acc = \frac{VP + VN}{FP + FN + VP + VN}$$

- Precision

$$Prec = \frac{VP}{VP + FP}$$

- Recall (Rappel)

$$Recall = \frac{VP}{VP + FN}$$



F Score

- F Score

$$F_\beta = (1 + \beta^2).\frac{Prec.Recall}{(\beta^2.Prec) + Recall}$$

Cette mesure permet de faire varier l'importance que l'on accorde à la Précision ou au Rappel.
Un β plus important $=$ avantage au Recall

En Python

```
#Données exemple
Yvrai = ['bleu', 'rouge', 'bleu', 'bleu', 'rouge', 'bleu', 'bleu', 'rouge']
Ypred = ['bleu', 'rouge', 'bleu', 'bleu', 'bleu', 'bleu', 'rouge', 'bleu']

# Métriques disponibles dans sklearn.metrics
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

confusion_matrix(Yvrai, Ypred)
accuracy_score(Yvrai, Ypred)
precision_score(Yvrai, Ypred, pos_label='rouge') #attention au label
recall_score(Yvrai, Ypred, pos_label='rouge') #attention au label
```